

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN ENXEÑARÍA DE COMPUTADORES



Sistema de captura y procesado de datos IMU en tiempo real para estimación de movimiento vertical

Estudiante: Mateo Rivela Santos

Dirección: Dirección por asignar

A Coruña, abril de 2026.

A mi familia, por su apoyo constante en cada etapa de este camino, y a todas las personas que me animaron a seguir aprendiendo cuando el resultado no salía a la primera.

Agradecimientos

Agradezco a mi entorno personal por la paciencia y el apoyo durante el desarrollo de este trabajo. Agradezco también al profesorado de la FIC por la base técnica y metodológica que ha hecho posible este proyecto, y a las personas que me ayudaron con pruebas, validaciones y revisión de resultados.

Resumen

Este trabajo presenta el desarrollo de un producto software móvil para captura y procesamiento de datos inerciales, con Android como plataforma ubicua principal. El sistema se diseña con dos modos de operación: modo autónomo (captura y procesamiento en el propio teléfono) y modo pantalla (visualización de resultados procesados por una plataforma externa como ESP32+MPU6050).

La solución implementada integra captura de sensores, normalización temporal, cálculo de métricas de oleaje y visualización en una misma aplicación, además de exportación de sesiones con parámetros y resultados. El algoritmo de procesamiento se utiliza como componente técnico del producto, no como fin en sí mismo.

Los resultados obtenidos muestran que la arquitectura de plataforma es viable y extensible, y permiten definir una comparativa homogénea entre Android y ESP32 para evaluar rendimiento, robustez y limitaciones en escenarios equivalentes.

Abstract

Este traballo presenta o desenvolvemento dun produto software móbil para captura e procesado de datos inerciais, con Android como plataforma ubicua principal. O sistema deseñouse con dous modos de operación: modo autónomo (captura e procesado no propio teléfono) e modo pantalla (visualización de resultados procesados por unha plataforma externa como ESP32+MPU6050).

A solución integra captura de sensores, normalización temporal, cálculo de métricas de ondata, visualización e exportación de sesións con parámetros e resultados nunha mesma aplicación. O algoritmo de procesado emprégase como compoñente técnico do produto e non como obxectivo final por si mesmo.

Os resultados mostran que a arquitectura de plataforma é viable e extensible, e permiten definir unha comparativa homoxénea entre Android e ESP32 para avaliar rendemento, robustez e limitacións en escenarios equivalentes.

Palabras clave:

- Unidad de medida inercial (IMU)
- Android
- Plataforma ubicua
- Producto software
- Procesado on-device
- Arquitectura móvil
- Comparativa Android-ESP32

Keywords:

- Unidade de medida inercial (IMU)
- Android
- Plataforma ubicua
- Produto software
- Procesado on-device
- Arquitectura móbil
- Comparativa Android-ESP32

Índice general

1	Introducción	1
1.1	Motivación del problema	1
1.2	Objetivos	2
1.2.1	Objetivo general	2
1.2.2	Objetivos específicos	2
1.3	Alcance y limitaciones	3
1.4	Metodología de desarrollo	4
1.5	Estructura de la memoria	4
2	Estado del arte y contexto tecnológico	6
2.1	Android como plataforma de sensado de bajo coste	6
2.1.1	Límites de una solución móvil pura	7
2.2	Nodos embebidos para adquisición dedicada	7
2.3	Arquitecturas híbridas móvil+embebido	8
2.4	Backend y operación remota	8
2.5	Contrato de datos e interoperabilidad	9
2.6	Posicionamiento del trabajo	9
3	Análisis del problema, requisitos, planificación y presupuesto	10
3.1	Descripción general del sistema a desarrollar	10
3.2	Requisitos funcionales	10
3.3	Requisitos no funcionales	11
3.4	Restricciones técnicas	11
3.5	Casos de uso principales	11
3.6	Enfoque de trabajo	12
3.7	Fases del proyecto	12
3.8	Planificación de operación remota (borrador)	13
3.9	Plan de pruebas y control de riesgos	13

3.10	Material y medios necesarios	14
3.11	Presupuesto orientativo	14
4	Diseño de la plataforma	15
4.1	Enfoque metodológico de diseño	15
4.2	Arquitectura funcional	15
4.2.1	Arquitectura física	15
4.2.2	Arquitectura lógica y software	16
4.3	Diseño de contratos de interfaz	16
4.3.1	Contrato de sesión	16
4.3.2	Contrato de control	17
4.3.3	Contrato de explotación	17
4.4	Diseño del subsistema de adquisición	17
4.4.1	Adquisición en modo móvil	17
4.4.2	Adquisición en modo embebido	17
4.4.3	Almacenamiento de sesiones	18
4.5	Diseño del subsistema de comunicación	18
4.6	Diseño del backend	18
4.6.1	Criterios de diseño de backend	18
4.6.2	Estrategia de evolución tecnológica	18
4.7	Diseño del flujo Android	20
4.8	Flujo de datos extremo a extremo	20
4.9	Diseño de comparativa entre plataformas	20
4.10	Diseño del subsistema de procesamiento	22
4.11	Diseño hardware del sistema embebido	22
5	Implementación del sistema	23
5.1	Visión general del estado actual	23
5.2	Implementación del modo móvil en Android	23
5.2.1	Captura y persistencia local	23
5.2.2	Gestión de sesiones	24
5.3	Implementación del modo ESP32	24
5.3.1	Flujo de operación embebida	24
5.3.2	Firmware del ESP32	24
5.4	Contrato de sesión y formato de datos	24
5.5	Visualización y explotación en la app	25
5.6	Diseño software y desacoplamiento	25
5.7	Estado de backend remoto	26

5.8	Herramientas de soporte de ingeniería	26
5.9	Limitaciones del estado implementado	26
6	Validación y evaluación	27
6.1	Metodología de validación	27
6.2	Escenarios de prueba	27
6.3	Configuración de pruebas	28
6.4	Métricas de evaluación	28
6.5	Plantilla de comparativa móvil vs embebido	29
6.6	Criterios de aceptación	29
6.7	Resultados	29
6.7.1	Resultados del modo móvil	29
6.7.2	Resultados del modo embebido	29
6.7.3	Resultados de integración por contratos	30
6.7.4	Comparación de operación entre modos	30
6.8	Discusión	31
6.8.1	Limitaciones del trabajo	31
6.8.2	Lecciones aprendidas	31
7	Conclusiones	32
7.1	Conclusiones principales	32
7.2	Conclusión metodológica	32
7.3	Implicaciones prácticas	33
7.4	Aportaciones	33
7.5	Trabajo futuro	33
7.5.1	Línea prioritaria inmediata	33
A	Material adicional	35
A.1	Scripts principales	35
A.2	Comandos de referencia	35
A.2.1	Captura	35
A.2.2	Procesado rápido sin gráfica	35
A.2.3	Procesado paso a paso con validación temporal	35
A.3	Datos de prueba	36
	Bibliografía	38

Índice de figuras

1.1	Esquema conceptual del problema abordado por el sistema.	2
3.1	Casos de uso principales del sistema.	12
4.1	Arquitectura funcional del sistema con Android como plataforma central de operación.	16
4.2	Arquitectura física del sistema y conexiones entre subsistemas.	16
4.3	Flujo Android: captura/operación, gestión de sesiones y visualización.	19
4.4	Flujo de datos extremo a extremo del sistema.	20
4.5	Flujo de comparativa entre nodo móvil y nodo embebido con contratos comunes.	21
4.6	Esquema del subsistema embebido y conexiones principales.	22
5.1	Vista de apoyo utilizada para revisión de sesiones y depuración del flujo de datos.	25
6.1	Escenarios de prueba del sistema.	28
6.2	Resultados representativos del modo móvil.	30
6.3	Resultados representativos del modo embebido.	30
6.4	Validación de contratos de integración entre subsistemas.	30
6.5	Comparación operativa entre modo móvil y modo embebido.	31

Índice de cuadros

2.1	Posicionamiento resumido del trabajo frente a enfoques habituales.	9
6.1	Plantilla de comparación entre modo móvil y modo embebido.	29

Introducción

ESTE trabajo fin de grado se centra en una idea principal: usar el teléfono móvil como plataforma ubicua para capturar y procesar datos inerciales, y demostrar su viabilidad en un caso de uso de caracterización de oleaje. El valor del sistema no está en un sensor concreto, sino en el producto software: su arquitectura de plataforma, su operación en distintos modos y su capacidad de despliegue práctico.

1.1 Motivación del problema

En la actualidad, existen multitud de problemas relacionados con la monitorización de movimientos en general, ya sea aceleración, orientación o posición, que pueden ser abordados con sensores inerciales.

El fácil acceso a datos inerciales en condiciones reales, sin necesidad de equipamiento especializado, abre la puerta a aplicaciones de monitorización de estado del mar, navegación, salud o actividad física. Lo que se pretende con este TFG es explorar esta posibilidad usando algo tan común como un teléfono móvil y demostrar que, con un tratamiento de señal adecuado, es posible obtener métricas útiles a partir de datos inerciales brutos.

Gracias a la ubicuidad de los smartphones, la capacidad de transmitir información desde ubicaciones relativamente remotas y la potencia de procesamiento que ofrecen, se pueden desarrollar multitud de aplicaciones que aprovechen esos datos para caracterizar el entorno o el estado de un sistema físico. En este caso, se ha elegido el oleaje como caso de uso representativo, ya que es un fenómeno complejo que puede ser monitorizado con sensores inerciales y tiene aplicaciones prácticas en navegación, meteorología y ocio.

En este contexto, los sensores inerciales permiten explorar una alternativa ligera frente a instrumentación oceanográfica más compleja o costosa, siempre que el tratamiento de señal y la arquitectura global del sistema sean adecuados. También hay que considerar que dependiendo de la aplicación concreta, la calidad que permiten los smartphones pueden o no

pueden ser suficientes, por lo que es importante validar esta aproximación y entender sus limitaciones.



Figura 1.1: Esquema conceptual del problema abordado por el sistema.

[TEXTO PROVISIONAL] En este trabajo se adopta como método principal un enfoque tipo OpenMetBuoy: estimación espectral mediante Welch, conversión de espectro de aceleración a elevación y cálculo de métricas de oleaje (H_s , T_z , T_c). La implementación se orienta a ejecución *on-device* en Android, con una fase posterior de comparación frente a ESP32+MPU6050.

1.2 Objetivos

1.2.1 Objetivo general

Diseñar e implementar una arquitectura modular de captura y gestión de datos inerciales, con Android como plataforma principal de operación y con capacidad de integración con un nodo embebido basado en ESP32+IMU.

1.2.2 Objetivos específicos

- Implementar en Android un modo autónomo de captura, guardado y gestión de sesiones.
- Implementar un modo de integración con ESP32 para consulta de estado, control de captura y descarga de sesiones.
- Definir contratos comunes de sesión, control y explotación para unificar ambos modos de operación.
- Estructurar el sistema en capas desacopladas (adquisición, comunicación, backend y visualización/control).
- Diseñar la integración con mecanismos de comunicación externa y acceso remoto a datos, sin depender de tecnologías concretas.

- Establecer un plan de validación centrado en funcionalidad, integración y operación del sistema.

1.3 Alcance y limitaciones

El alcance de la memoria es el diseño e implementación de una arquitectura operativa para adquisición y gestión de datos inerciales, no el cierre de una solución algorítmica definitiva para un único dominio de aplicación.

Dentro de este alcance se cubren la operación local en Android, la integración con un nodo embebido ESP32+IMU y la definición de interfaces comunes para evolucionar la arquitectura de forma controlada.

Las limitaciones principales consideradas son:

- **Variabilidad de sensores y temporización entre dispositivos Android.** Hay que tener en cuenta que diferentes modelos de teléfono presentan diferencias de hardware, calibración interna y frecuencia efectiva de muestreo. Esto claramente puede introducir variaciones en los datos capturados que si comprobamos la respuesta de ambos dispositivos incluso bajo condiciones similares van a mostrar resultados distintos. Esto debemos tenerlo en cuenta en la comparación entre sesiones, que requiere normalización y control de metadatos.
- **Estado iterativo de la integración hardware en el subsistema embebido.** El hardware siempre va a presetar una serie de problemas añadidos. En este caso la integración ESP32+IMU depende de aspectos prácticos de cableado, alimentación, estabilidad del bus , además de la introducción de estas en un entorno de prueba realista y tener en cuenta ese entorno para la propia construcción del sistema ,aunque para testear el sistema se ha optado por un entorno controlado, es importante tener en cuenta que el entorno real de uso puede introducir variaciones adicionales que deben ser consideradas en la evolución futura del sistema. Como consecuencia, parte del trabajo embebido se desarrolla de forma iterativa, combinando implementación y diagnóstico técnico.

[TEXTO PROVISIONAL]

- **Integración de backend cloud aún en fase de consolidación.** Aunque la arquitectura contempla sincronización y acceso remoto, la capa cloud todavía no está cerrada como componente definitivo del sistema. En esta fase, el foco se mantiene en estabilizar contratos de datos e interoperabilidad antes de fijar una solución de despliegue final.
- **Dependencia de campañas de prueba para cerrar validación comparativa completa.** La comparación entre plataformas requiere campañas repetibles con protocolo

homogéneo de captura y operación. Mientras dichas campañas no estén completamente cerradas, las conclusiones comparativas deben considerarse provisionales y sujetas a validación adicional.

- **Restricciones de tiempo y alcance propias de un TFG.** El proyecto prioriza la definición arquitectónica, la implementación funcional y la trazabilidad técnica frente a una industrialización completa. Esto implica posponer algunas mejoras de robustez avanzada, automatización extensa y validación a gran escala para trabajo futuro.

1.4 Metodología de desarrollo

La memoria se apoya en cuatro marcos metodológicos complementarios:

- **metodología de desarrollo incremental**, para construir el sistema por fases;
- **enfoque modular ciberfísico**, para desacoplar subsistemas e interfaces;
- **ingeniería de integración**, para validar contratos comunes entre nodos;
- **metodología de validación operativa**, para evaluar funcionalidad y robustez de uso.

En términos prácticos, la metodología aplicada sigue una secuencia incremental y orientada a validación:

1. Definición de arquitectura por capas y contratos de sesión/control.
2. Consolidación del modo móvil en Android como flujo base.
3. Integración del modo embebido ESP32 en el flujo de operación de la app.
4. Evaluación e integración preliminar de mecanismos de supervisión y acceso externo.
5. Validación de interoperabilidad y documentación de evolución futura.

1.5 Estructura de la memoria

La memoria se organiza de la siguiente forma:

- En el Capítulo 2 se resume el estado del arte estrictamente necesario para justificar decisiones de producto.
- En el Capítulo 3 se describe la metodología de desarrollo, la planificación y el presupuesto de trabajo.

- En el Capítulo 4 se define la arquitectura funcional por capas y los contratos de interfaz.
- En el Capítulo 5 se documenta el estado real de implementación en modo móvil y embebido.
- En el Capítulo 6 se presenta la validación funcional, de integración y operativa del sistema.
- En el Capítulo 7 se recogen conclusiones, aportaciones y trabajo futuro.
- En el Apéndice A se incluye material complementario.

Estado del arte y contexto tecnológico

ESTE capítulo pretende situar el trabajo de fin de grado en el contexto de las tecnologías y enfoques existentes con comportamiento de IOT mediante android.

2.1 Android como plataforma de sensado de bajo coste

El desarrollo de IOT tiene un amplio espectro de enfoques y variedad de desarrollos, desde soluciones de hardware dedicado, desarrollo de menos especializado y de código abierto, hasta soluciones comerciales cerradas. En este contexto, el uso de smartphones como plataforma de iot es una respuesta inteligente ya que si analizamos las características de los smartphones modernos, vemos que cuentan con una amplia variedad de sensores integrados, incluyendo acelerómetros, giroscopios y magnetómetros, que pueden ser utilizados para capturar datos inerciales sin necesidad de hardware adicional. Además, los smartphones ofrecen capacidades de procesamiento y conectividad que permiten no solo la captura de datos, sino también su procesamiento local y transmisión a servidores o nubes para análisis posterior.

Teniendo esto en cuenta, el enfoque de este TFG se centra en aprovechar estas capacidades de los smartphones para desarrollar una solución de captura y gestión de datos inerciales que sea accesible, fácil de usar y que pueda ser desplegada rápidamente.

Frente a soluciones IoT dedicadas, Android permite iterar rápido y desplegar prototipos funcionales sin depender de hardware específico desde el primer día.

[TEXTO PROVISIONAL]

En términos de producto, esta decisión aporta tres ventajas claras:

- acceso inmediato a sensores inerciales integrados;

- interfaz de usuario completa para operación, diagnóstico y revisión;
- almacenamiento local y exportación sin infraestructura adicional.

2.1.1 Límites de una solución móvil pura

El enfoque móvil evidentemente presenta claras limitaciones como pueden ser:

- **Variabilidad entre dispositivos:** Existen innumerables modelos de smartphones en el mercado, cada uno con diferentes sensores, calibraciones, procesamiento, además de esto diferentes versiones de android, incluso no tiene porque ser android. Al tener esto en cuenta no podemos esperar un comportamiento estandar especialmente si se espera un nivel de precision minimamente profesional, por lo que es importante considerar esta variabilidad en el diseño del sistema y en la interpretación de los datos capturados.
- **control limitado del hardware sensorial:** Los smartphones no estan pensados para ser configurables a nivel de hardware, lo que implica que aun teniendo acceso, en nuestro caso , de los sensores inerciales, no podemos controlar ciertas características como la frecuencia de muestreo, sensibilidad , etc. Evidentemente si comparamos esto con una solución de hardware dedicado, es una desventaja clara . Sin embargo, nosotros no buscamos una solución que nos lleve a una medición de precision profesional, sino una solución que nos permita capturar datos de forma sencilla , con un nivel de calidad suficiente para ciertos casos de uso y poder desarrollar una arquitectura que en un futuro pueda ser integrada con mejores prestaciones de hardware.

[TEXTO PROVISIONAL]

Por ello, la arquitectura del TFG no se plantea como solución móvil cerrada, sino como sistema extensible a nodos embebidos.

Además de estas limitaciones técnicas, también hay que considerar aspectos prácticos de uso: la duración de la batería, la ergonomía de uso en entornos marinos o la resistencia a condiciones adversas pueden ser factores limitantes para el despliegue de una solución móvil pura en escenarios reales. Estos aspectos también se tienen en cuenta en el diseño de la arquitectura, que busca ser flexible para adaptarse a diferentes escenarios de uso y permitir la integración de hardware dedicado cuando sea necesario.

2.2 Nodos embebidos para adquisición dedicada

Hay dos razones principales por las que se opta por un nodo embebido además de la solución móvil:

- **Comparación directa de captura.** La idea de usar nodos embebidos es obtener una comparación directa de la captura que obtenemos con el dispositivo móvil a la captura que obtenemos con el nodo embebido, de esta forma tenemos un punto de referencia para evaluar la calidad de nuestra oferta en comparación a una solución de hardware dedicado, que conllevaría un coste, complejidad y tiempo de desarrollo mayor.
- **Caso de uso extra** Además de esto, el tener más de un nodo de captura de datos es una posible aplicación de otras aplicaciones, en nuestro caso, el caso de uso era el de caracterización de oleaje en un punto concreto, pero si tenemos dos nodos, podríamos por ejemplo ponerlos en dos puntos de un barco y obtener los datos de ambos puntos para obtener algún tipo de análisis.

En este trabajo, esa línea se representa con ESP32+IMU, usando el ESP32 como nodo de captura/comunicación y Android como interfaz de operación.

2.3 Arquitecturas híbridas móvil+embebido

El patrón arquitectónico relevante para este TFG es híbrido:

- nodo móvil para operación rápida y despliegue inmediato;
- nodo embebido para adquisición dedicada cuando el escenario lo requiera;
- contrato de datos común para evitar bifurcaciones por plataforma.

La comparación entre móvil y embebido no se entiende como sustitución directa, sino como análisis de compromisos entre simplicidad operativa, control técnico y mantenibilidad.

2.4 Backend y operación remota

En sistemas de captura distribuidos, el backend condiciona tanto la velocidad de desarrollo como la robustez operativa. Para el alcance del TFG, tiene sentido distinguir dos estrategias:

- **Backend gestionado**, orientado a acelerar sincronización, persistencia y consulta de sesiones;
- **Mensajería de telemetría**, orientada a escenarios de control/estado en tiempo real.

En este TFG, la arquitectura se define para poder integrar cualquiera de estas alternativas sin acoplar el diseño a una tecnología específica.

2.5 Contrato de datos e interoperabilidad

En trabajos de integración extremo a extremo, la clave no es solo capturar datos, sino mantener trazabilidad entre nodos, sesiones y resultados. Por ello, el estado del arte aplicado en este TFG enfatiza contratos de datos explícitos:

- contrato de sesión (identidad de nodo, timestamps, frecuencia efectiva, payload, meta-datos);
- contrato de control (start, stop, set_config, status);
- contrato de explotación (listado, detalle y exportación de sesiones).

Este enfoque reduce acoplamiento y permite evolucionar backend o transporte sin rehacer la lógica de adquisición.

2.6 Posicionamiento del trabajo

La aportación del TFG se sitúa en la integración de arquitectura, no en un único componente aislado. El trabajo se posiciona como una propuesta de sistema ciberfísico modular que cubre adquisición, comunicación, persistencia y operación.

Aspecto	Enfoques frecuentes	Enfoque del TFG
Plataforma base	App móvil o nodo embebido por separado	Arquitectura híbrida Android+ESP32
Persistencia	Almacenamiento local o backend ad-hoc	Persistencia desacoplada con alternativas evaluables
Integración	Piezas aisladas (captura, visualización, etc.)	Flujo extremo a extremo con contratos comunes
Evolución técnica	Cambios con alto acoplamiento	Diseño modular con sustitución de capas

Cuadro 2.1: Posicionamiento resumido del trabajo frente a enfoques habituales.

Análisis del problema, requisitos, planificación y presupuesto

LA ejecución del TFG se plantea como un desarrollo incremental de arquitectura, orientado a obtener un sistema funcional por fases y a reducir riesgo técnico en integración.

3.1 Descripción general del sistema a desarrollar

El sistema se define como una arquitectura ciberfísica para captura y gestión de datos inerciales. La propuesta integra adquisición, comunicación, persistencia y visualización/control bajo una estructura modular, de forma que Android y el nodo embebido puedan interoperar sin depender de implementaciones rígidas.

3.2 Requisitos funcionales

Los requisitos funcionales principales del sistema son:

- capturar datos inerciales desde nodo móvil Android;
- capturar o descargar sesiones desde nodo embebido ESP32+IMU;
- almacenar sesiones con metadatos trazables;
- visualizar estado de nodo, sesiones y resultados de forma operativa;
- exportar sesiones para análisis externo;
- soportar comandos básicos de control remoto (inicio, parada, estado y configuración).

3.3 Requisitos no funcionales

Además de la funcionalidad visible, el diseño debe cumplir:

- modularidad entre adquisición, comunicación, backend y visualización;
- mantenibilidad para evolucionar backend/transporte sin rehacer el sistema;
- bajo coste de despliegue en entorno de TFG;
- trazabilidad de sesiones y de parámetros de ejecución;
- facilidad de operación en laboratorio y escenarios remotos.

3.4 Restricciones técnicas

Las restricciones más relevantes del proyecto son:

- heterogeneidad de sensores y temporización en Android;
- limitaciones de recursos y estabilidad de integración en ESP32;
- necesidad de definir contratos de datos comunes entre nodos;
- tiempo de desarrollo limitado para cerrar integración extremo a extremo;
- incertidumbre sobre qué transporte remoto aporta mejor equilibrio en este alcance.

3.5 Casos de uso principales

Los casos de uso que estructuran el sistema son:

- captura y registro local desde Android;
- operación de captura desde ESP32 con supervisión desde la app;
- consulta histórica y reprocesado de sesiones;
- comparación de sesiones entre nodos/dispositivos;
- configuración remota y consulta de estado.

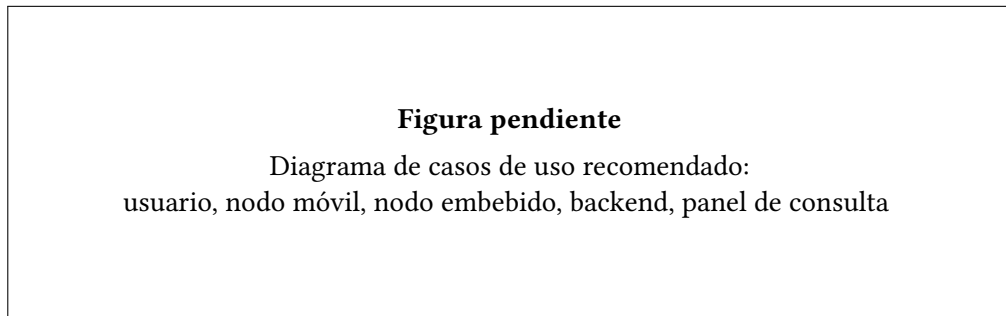


Figura 3.1: Casos de uso principales del sistema.

3.6 Enfoque de trabajo

Se adopta una **metodología incremental e iterativa**. La razón principal es separar riesgos por capas: primero análisis del problema y diseño arquitectónico, después integración entre nodos y finalmente validación operativa.

Este enfoque permite validar decisiones con entregables funcionales y evitar bloqueos por dependencias tempranas de infraestructura. El trabajo parte de un análisis inicial del problema, del estado del arte y de las tecnologías disponibles para definir la arquitectura final del sistema.

De forma resumida, la estrategia por bloques funcionales es:

1. captura y persistencia de sesión en local;
2. interfaz operativa Android y gestión de sesiones;
3. integración local con ESP32 vía interfaz de red;
4. incorporación progresiva de supervisión y acceso externo a datos;
5. evaluación final del sistema y propuestas de mejora.

3.7 Fases del proyecto

- **Fase 1.** Análisis de trabajos relacionados en sensores inerciales, análisis de movimiento y arquitecturas de captura de datos.
- **Fase 2.** Estudio de limitaciones y características de sensores IMU y técnicas de análisis aplicables.
- **Fase 3.** Definición de la arquitectura del sistema y de la interacción entre módulos.
- **Fase 4.** Implementación de captura y gestión de datos en diferentes plataformas.

- **Fase 5.** Evaluación del sistema desarrollado y análisis de comportamiento, limitaciones y mejoras.
- **Fase 6.** Aplicación del sistema al caso práctico propuesto.
- **Fase 7.** Documentación final y presentación.

3.8 Planificación de operación remota (borrador)

La operación remota se plantea como evolución del sistema y no como requisito cerrado desde el inicio. En el estado actual, el objetivo es dejar una base operativa con sincronización de sesiones y estado de nodos.

Para ello se propone:

- **Implementación base:** integración con mecanismo de almacenamiento/sincronización externo según viabilidad del entorno de pruebas.
- **Sincronización mínima:** metadatos de sesión, estado de nodo y referencias a ficheros.
- **Control básico:** comandos start/stop/set_config/status bajo contrato común.
- **Evolución opcional:** evaluar alternativas de comunicación si mejoran latencia, robustez o escalabilidad.

Este planteamiento evita comprometer decisiones de infraestructura que no estén justificadas por evidencia experimental.

3.9 Plan de pruebas y control de riesgos

Los riesgos principales y su mitigación son:

- **Desalineación entre nodos:** fijar contrato de sesión y validarlo en ambos modos.
- **Inestabilidad de integración embebida:** mantener diagnósticos de conexión, bus y captura.
- **Acoplamiento excesivo al backend:** desacoplar capa de comunicación y persistencia.
- **Sobrecarga de alcance:** priorizar MVP funcional antes de añadir extensiones opcionales.

3.10 Material y medios necesarios

Los medios necesarios para ejecutar el proyecto son:

- PC para desarrollo, pruebas y análisis de resultados.
- Dispositivo móvil Android con sensores inerciales.
- Plataforma embebida para captura de datos con conectividad inalámbrica.
- Sensor IMU externo (acelerómetro y/o giróscopo).
- Herramientas de desarrollo software y entornos de programación.
- Herramientas para almacenamiento, visualización y análisis de datos.

3.11 Presupuesto orientativo

El trabajo prioriza bajo coste de hardware:

- teléfono Android como nodo principal;
- placa ESP32 disponible para integración embebida;
- IMU MPU6050 para pruebas de adquisición dedicada.

El coste principal se concentra en tiempo de desarrollo, integración y validación del sistema extremo a extremo.

Diseño de la plataforma

EL diseño se centra en una arquitectura modular, con Android como interfaz principal y capacidad de integración con un nodo embebido basado en ESP32+IMU.

4.1 Enfoque metodológico de diseño

El sistema se diseña como un sistema ciberfísico por capas. Este enfoque permite separar responsabilidades y evolucionar componentes sin rehacer el producto completo.

Desde esta perspectiva, la plataforma se organiza en cuatro capas:

- **adquisición:** captura de datos inerciales en nodos móvil y embebido;
- **comunicación:** transporte de comandos, estado y sesiones;
- **backend:** persistencia y sincronización de información;
- **interfaz/control:** operación del sistema y explotación de resultados.

4.2 Arquitectura funcional

La arquitectura global contempla dos modos operativos:

- **Modo móvil:** Android captura, almacena y explota sesiones de forma autónoma.
- **Modo embebido:** ESP32 adquiere y Android supervisa, descarga y gestiona sesiones.

4.2.1 Arquitectura física

El despliegue físico básico se compone de teléfono Android, nodo ESP32 y IMU externa. Según el escenario, la adquisición puede realizarse íntegramente en el teléfono o delegarse en el subsistema embebido.

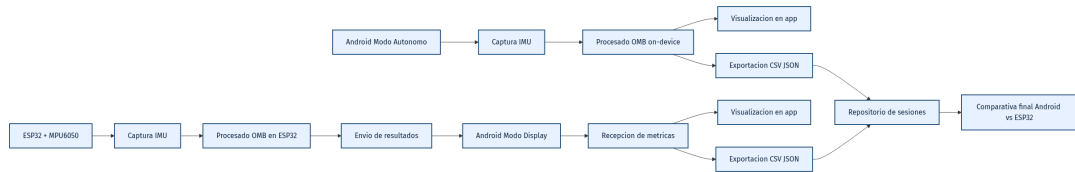


Figura 4.1: Arquitectura funcional del sistema con Android como plataforma central de operación.

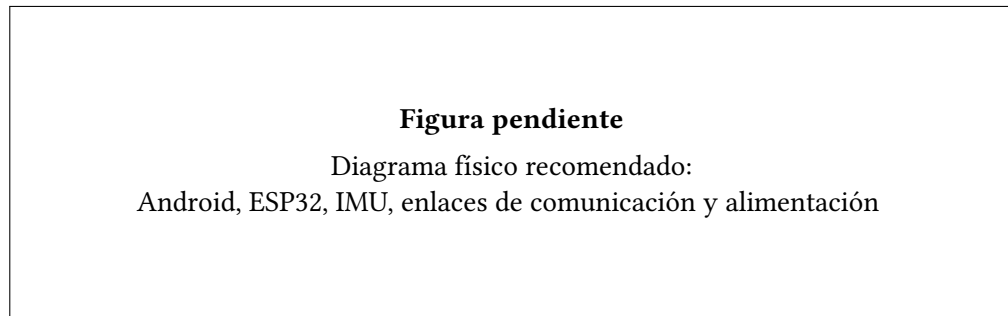


Figura 4.2: Arquitectura física del sistema y conexiones entre subsistemas.

4.2.2 Arquitectura lógica y software

A nivel lógico, el sistema se divide en módulos desacoplados:

- captura y normalización de muestras;
- persistencia y catálogo de sesiones;
- comunicación entre nodos;
- visualización, operación y exportación.

Esta separación facilita mantenimiento, pruebas y evolución de backend/transporte.

4.3 Diseño de contratos de interfaz

4.3.1 Contrato de sesión

La sesión es la unidad principal de intercambio entre subsistemas. El contrato de sesión incluye, como mínimo:

- `device_id`;
- `source` (móvil/embebido/simulado);

- `timestamps`;
- `fs` estimada;
- `payload` de muestras inerciales;
- metadatos de calidad y contexto.

4.3.2 Contrato de control

Para operación remota y coordinación de nodos se define un contrato mínimo de control:

- `start`;
- `stop`;
- `set_config`;
- `status`.

4.3.3 Contrato de explotación

La capa de explotación proporciona operaciones homogéneas de:

- listado de sesiones;
- detalle de sesión;
- exportación de datos.

4.4 Diseño del subsistema de adquisición

La adquisición se mantiene desacoplada del resto de capas para permitir cambios de nodo sin romper el flujo global.

4.4.1 Adquisición en modo móvil

Android registra sensores inerciales, crea sesiones locales y permite revisión inmediata de resultados. Este modo sirve como camino base del sistema por simplicidad operativa.

4.4.2 Adquisición en modo embebido

En modo embebido, ESP32 se encarga de la captura y Android actúa como consola de operación y descarga. La comunicación entre ambos se basa en un conjunto acotado de endpoints y estados de captura.

4.4.3 Almacenamiento de sesiones

El almacenamiento se diseña para trazabilidad y reutilización:

- persistencia local durante captura;
- metadatos de origen y configuración;
- compatibilidad con sincronización a backend.

4.5 Diseño del subsistema de comunicación

La capa de comunicación se diseña para soportar dos escenarios:

- **enlace local** Android-ESP32 para operación de laboratorio y pruebas de integración;
- **sincronización remota** con backend para consulta y operación desacoplada.

El objetivo es evitar dependencia fuerte de un único transporte.

4.6 Diseño del backend

4.6.1 Criterios de diseño de backend

El backend se diseña como capa intercambiable, con independencia de proveedor o tecnología concreta. En esta capa se contemplan las siguientes capacidades mínimas:

- persistencia de metadatos de sesión;
- almacenamiento de ficheros o exportaciones;
- sincronización de estado entre nodos.

4.6.2 Estrategia de evolución tecnológica

La selección tecnológica final del backend y del transporte de comunicaciones se plantea por criterios de validación práctica: latencia operativa, robustez de despliegue, mantenibilidad y coste. Este enfoque evita cerrar prematuramente la arquitectura a una única solución.

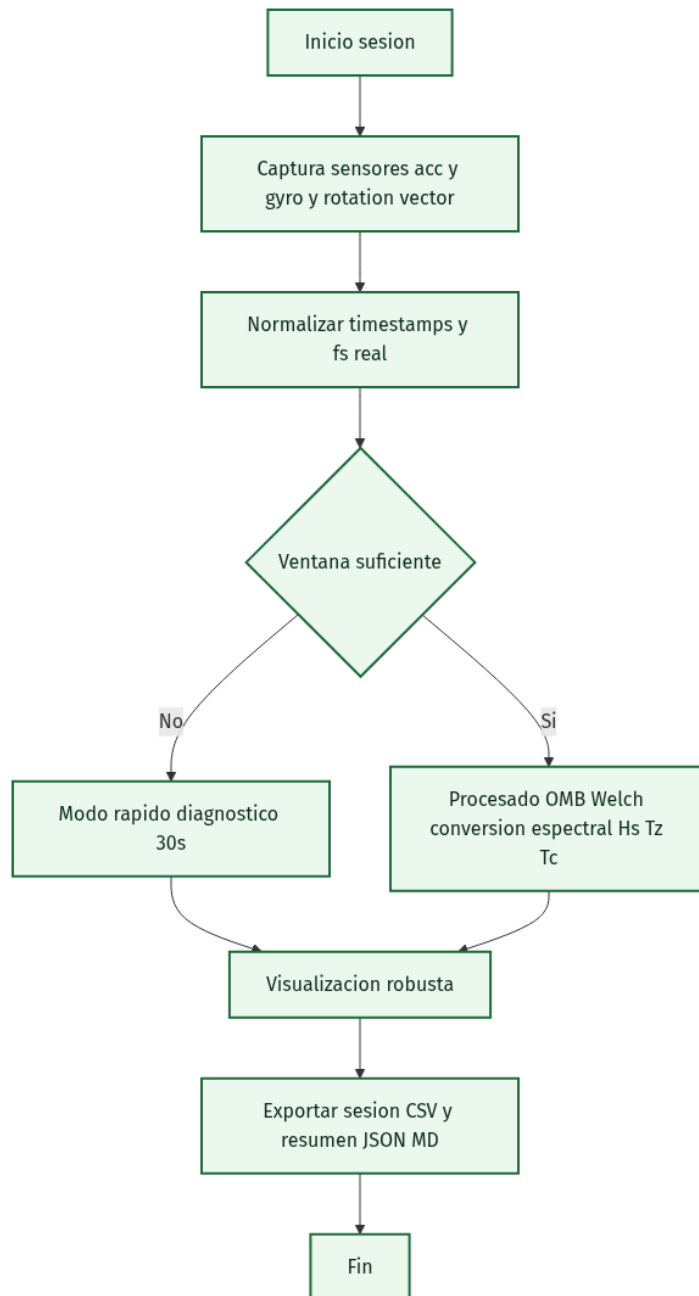


Figura 4.3: Flujo Android: captura/operación, gestión de sesiones y visualización.

4.7 Diseño del flujo Android

La app se diseña como interfaz de operación unificada:

- control de captura y estado de nodo;
- navegación por sesiones locales y externas;
- visualización de datos y exportación.

4.8 Flujo de datos extremo a extremo

El flujo completo del sistema se define así:

1. adquisición de muestras;
2. normalización y empaquetado de sesión;
3. persistencia local y/o sincronización remota;
4. recuperación para visualización, comparación y exportación.

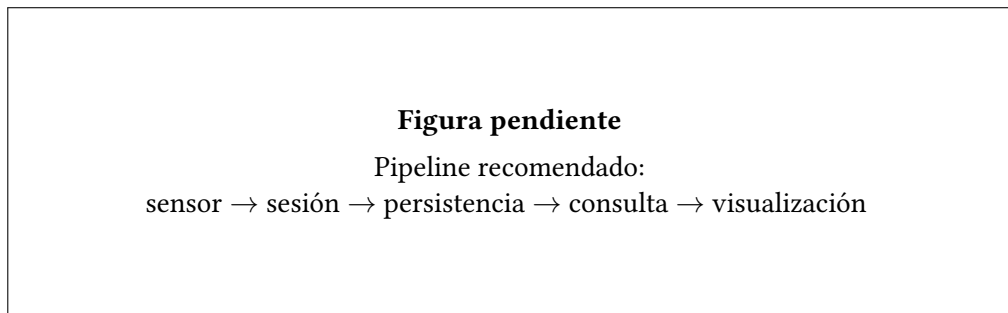


Figura 4.4: Flujo de datos extremo a extremo del sistema.

4.9 Diseño de comparativa entre plataformas

La comparativa se orienta a evaluar interoperabilidad y operación, no solo valores numéricos aislados. Los ejes principales son estabilidad de captura, trazabilidad de sesión, latencia operativa y robustez de integración.

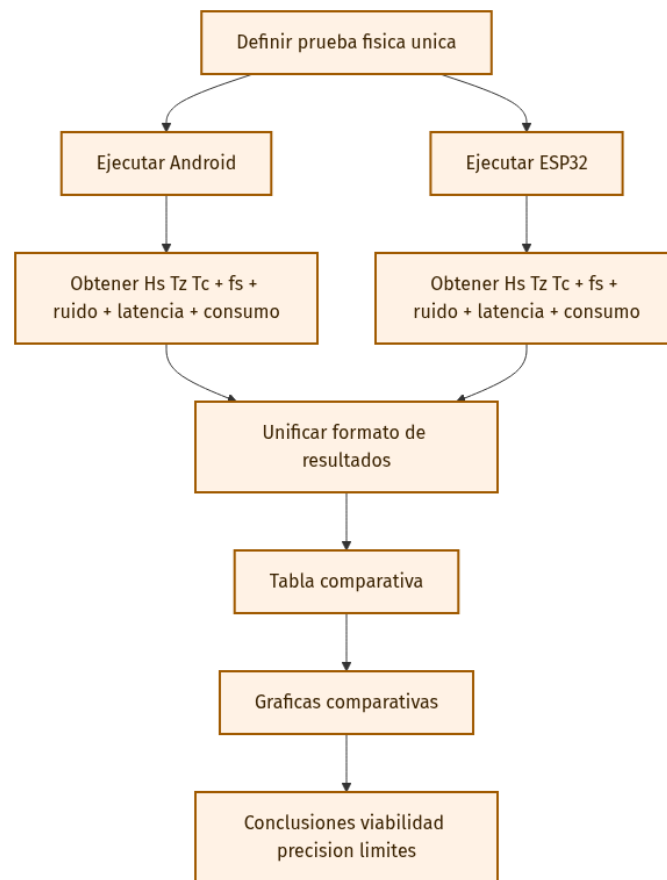


Figura 4.5: Flujo de comparativa entre nodo móvil y nodo embebido con contratos comunes.

4.10 Diseño del subsistema de procesado

El procesado se mantiene como bloque desacoplado de la captura. Esto permite reemplazar o evolucionar métodos de análisis sin alterar los contratos de adquisición y comunicación.

La arquitectura exige que cualquier bloque de procesado cumpla:

- entrada basada en contrato de sesión;
- salida estructurada y trazable;
- compatibilidad con ambos orígenes (móvil y embebido).

4.11 Diseño hardware del sistema embebido

El subsistema embebido se compone de ESP32, IMU y electrónica auxiliar de conexión/alimentación. Su papel en la arquitectura es doble: adquisición dedicada y validación de interoperabilidad con la app Android.

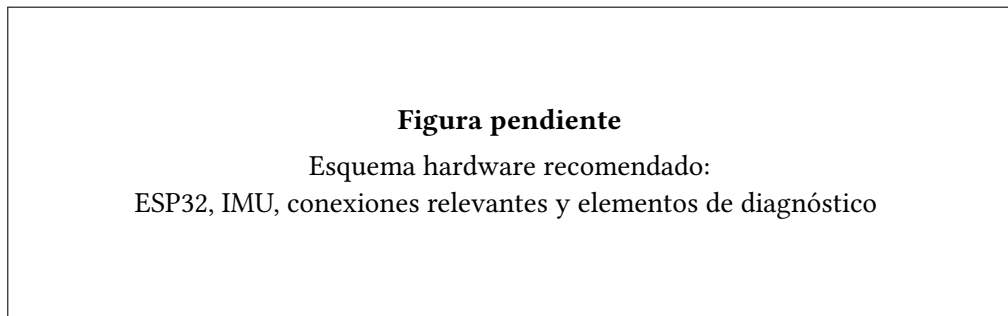


Figura 4.6: Esquema del subsistema embebido y conexiones principales.

Implementación del sistema

EN este capítulo se describe el estado de implementación del sistema según la arquitectura modular definida, diferenciando claramente lo implementado de las líneas de evolución.

5.1 Visión general del estado actual

La implementación actual cubre un flujo funcional extremo a extremo en entorno local:

- captura de datos inerciales en Android;
- gestión local de sesiones;
- integración con ESP32 para control de captura y descarga;
- visualización operativa de sesiones desde la aplicación.

La capa backend remota se mantiene como fase de evolución y no como componente cerrado en el estado actual.

5.2 Implementación del modo móvil en Android

El modo móvil registra sensores inerciales del teléfono y guarda las capturas como sesiones en almacenamiento de la app. Cada sesión incluye muestras temporales y metadatos mínimos para trazabilidad.

5.2.1 Captura y persistencia local

La app implementa captura con duración configurable y guardado automático en ficheros de sesión. Esta decisión permite trabajar sin infraestructura externa en etapas tempranas y facilita depuración rápida.

5.2.2 Gestión de sesiones

Se implementan operaciones de catálogo para listar, renombrar, eliminar y abrir sesiones. Esto convierte la app en una herramienta operativa de trabajo y no solo en una pantalla de captura puntual.

5.3 Implementación del modo ESP32

La integración embebida se implementa como flujo de operación desde Android contra el nodo ESP32. El intercambio actual se realiza por red local, con endpoints para verificación de estado, control de captura y descarga de datos.

5.3.1 Flujo de operación embebida

La app permite:

- comprobar conectividad con el nodo;
- iniciar/parar captura remota;
- descargar sesión generada en el ESP32;
- almacenar y visualizar la sesión de forma homogénea al modo móvil.

Este diseño mantiene una experiencia operativa consistente entre ambos orígenes de datos.

5.3.2 Firmware del ESP32

El firmware actual incluye funciones de red, estado de captura y descarga de sesiones. Además, incorpora soporte de diagnóstico y un modo de simulación para acelerar pruebas cuando la integración física del sensor no está cerrada.

5.4 Contrato de sesión y formato de datos

El sistema utiliza una sesión como unidad de trabajo compartida. En la implementación actual, el formato local de referencia está orientado a series temporales con metadatos básicos.

Se mantiene como criterio de diseño que la sesión contenga identidad de origen, base temporal y parámetros de captura suficientes para comparación y reprocesado.

5.5 Visualización y explotación en la app

La aplicación integra visualización de sesiones y panel de estado de operación. El objetivo principal de esta capa es permitir inspección rápida, trazabilidad de capturas y flujo de trabajo continuo (captura, revisión, gestión y exportación).

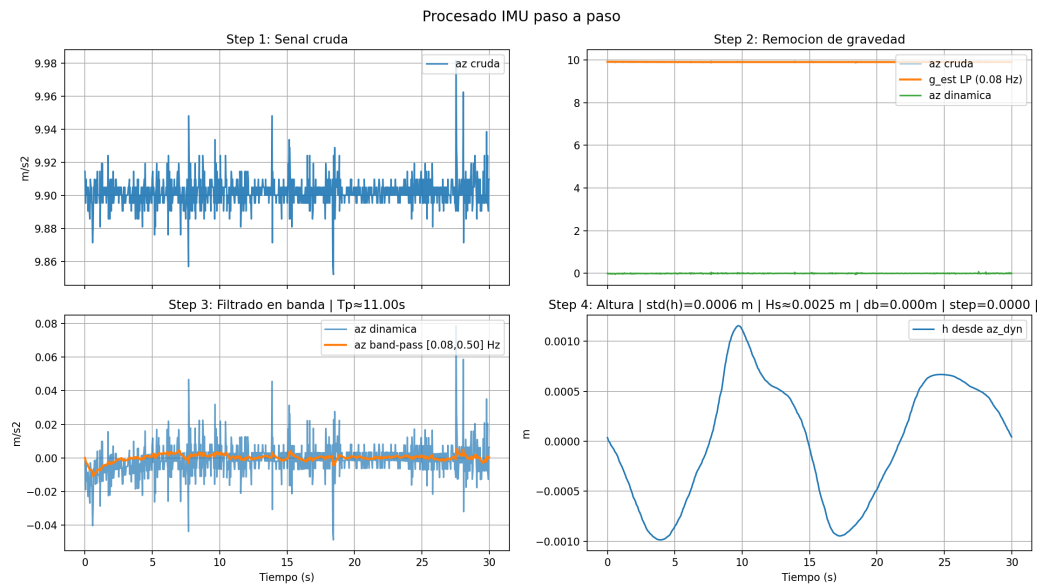


Figura 5.1: Vista de apoyo utilizada para revisión de sesiones y depuración del flujo de datos.

5.6 Diseño software y desacoplamiento

A nivel de implementación, se mantiene separación entre:

- captura;
- repositorio de sesiones;
- comunicación con nodo embebido;
- visualización y operación.

Esta separación reduce acoplamiento y facilita introducir backend remoto o cambios de transporte sin rehacer la aplicación.

5.7 Estado de backend remoto

La integración con backend cloud se considera una fase posterior del desarrollo. En esta etapa, se mantienen definidos los requisitos de capa remota (sincronización de sesiones, estado de nodos y acceso externo), dejando abierta la elección tecnológica final.

5.8 Herramientas de soporte de ingeniería

Durante el desarrollo se han utilizado scripts y utilidades de diagnóstico para acelerar verificación de captura, comunicación y consistencia de sesiones. Estas herramientas se consideran apoyo técnico de implementación y no sustituyen al flujo de producto principal en Android.

5.9 Limitaciones del estado implementado

En el estado actual del TFG, las limitaciones más relevantes son:

- dependencia de operación local para parte del flujo embebido;
- backend remoto todavía no integrado en la rama principal;
- necesidad de consolidar validación comparativa completa bajo protocolo homogéneo.

Estas limitaciones se asumen explícitamente como parte de un desarrollo incremental orientado a reducir riesgo técnico.

Validación y evaluación

LA validación se orienta a comprobar la viabilidad del sistema como arquitectura operativa y a medir el grado de madurez de integración entre sus capas.

6.1 Metodología de validación

La evaluación se diseña con enfoque de ingeniería aplicada, separando tres niveles:

- **validación funcional:** cada bloque cumple su responsabilidad;
- **validación de integración:** interoperabilidad entre Android, ESP32 y repositorio de sesiones;
- **validación operativa:** estabilidad del flujo de trabajo en escenarios de uso repetido.

El objetivo no es obtener un único resultado numérico, sino demostrar que el sistema funciona de forma trazable y mantenible.

6.2 Escenarios de prueba

Los escenarios principales considerados son:

- captura y gestión de sesión en modo móvil;
- operación remota local sobre ESP32 (estado, inicio/parada y descarga);
- consistencia del contrato de sesión entre orígenes móvil y embebido;
- robustez de operación tras múltiples iteraciones de captura y consulta.

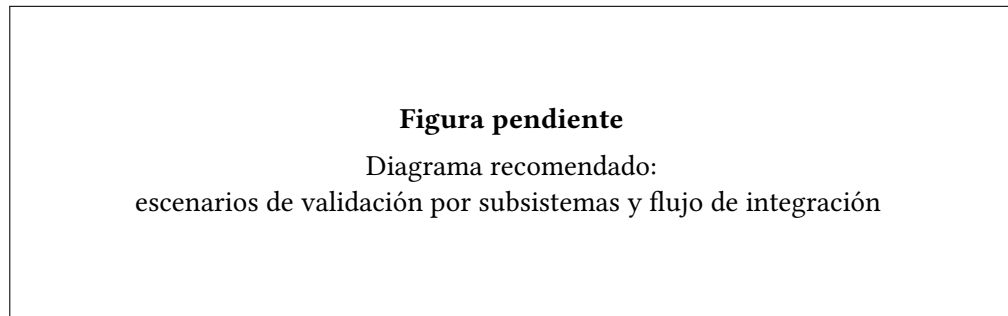


Figura 6.1: Escenarios de prueba del sistema.

6.3 Configuración de pruebas

Para cada prueba se documentan, como mínimo:

- nodo origen de los datos (móvil o embebido);
- parámetros de captura usados en la sesión;
- versión de app/firmware empleada;
- ruta de comunicación utilizada (local o sincronizada);
- resultado de operación (éxito, error o comportamiento degradado).

Esta documentación permite reproducibilidad y facilita análisis de incidencias.

6.4 Métricas de evaluación

Las métricas se centran en comportamiento del sistema:

- tasa de sesiones completadas sin error;
- latencia percibida en operaciones de control (start/stop/descarga);
- consistencia de metadatos y formato de sesión;
- estabilidad de conexión en modo embebido;
- esfuerzo operativo (pasos necesarios por flujo).

Aspecto	Nodo móvil (Android)	Nodo embebido (ESP32+IMU)	Criterio de evaluación
Flujo de captura	Local y directo	Remoto con control desde app	Compleitud de flujo
Persistencia	Sesión local en app	Descarga y guardado en app	Trazabilidad de sesión
Operación	Interacción simple	Mayor dependencia de integración	Robustez operativa
Estado/diagnóstico	Inmediato en interfaz	Requiere chequeo de conectividad	Tiempo de resolución
Evolución cloud	Integración prevista con backend	Integración prevista vía contratos comunes	Escalabilidad de arquitectura

Cuadro 6.1: Plantilla de comparación entre modo móvil y modo embebido.

6.5 Plantilla de comparativa móvil vs embebido

6.6 Criterios de aceptación

Se considerará validada la arquitectura en el alcance del TFG cuando:

- el flujo captura-guardado-consulta funcione de forma repetible;
- la integración Android-ESP32 permita control y descarga de sesiones;
- el formato de sesión sea consistente entre orígenes;
- se documenten límites operativos y decisiones de evolución pendientes.

6.7 Resultados

Este apartado se reserva para presentar resultados reales de campaña, distinguiendo claramente entre validación funcional, integración y operación.

6.7.1 Resultados del modo móvil

Incluir sesiones representativas de captura local, tiempos de operación y comportamiento de gestión de sesiones.

6.7.2 Resultados del modo embebido

Documentar resultados de conexión, control de captura y descarga de sesiones desde ESP32, incluyendo incidencias observadas.

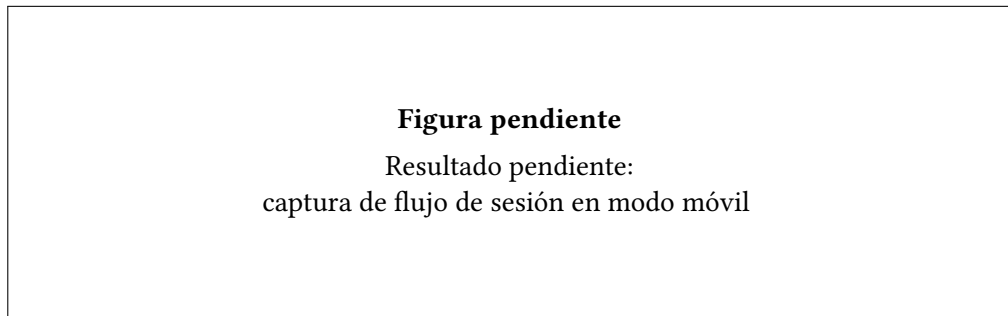


Figura 6.2: Resultados representativos del modo móvil.

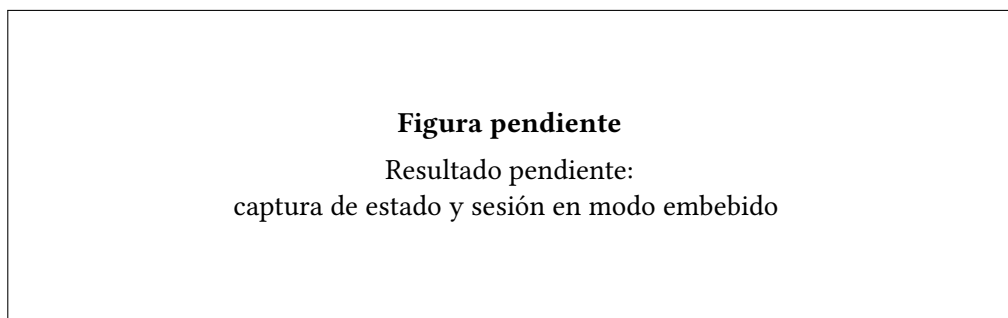


Figura 6.3: Resultados representativos del modo embebido.

6.7.3 Resultados de integración por contratos

Este bloque debe mostrar si los contratos de sesión y control se cumplen de forma homogénea y dónde aparecen incompatibilidades.

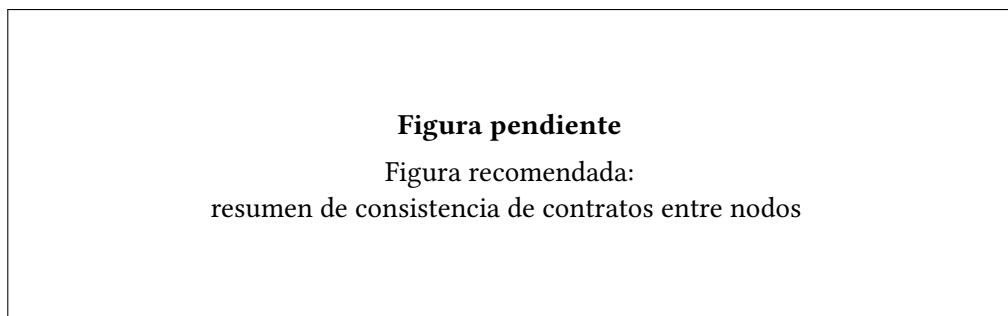


Figura 6.4: Validación de contratos de integración entre subsistemas.

6.7.4 Comparación de operación entre modos

La comparación debe centrarse en esfuerzo operativo, estabilidad de flujo y mantenibilidad, no solo en valores puntuales.

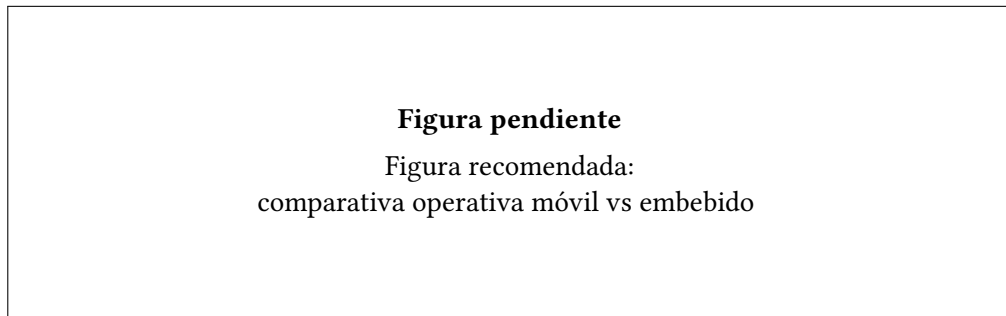


Figura 6.5: Comparación operativa entre modo móvil y modo embebido.

6.8 Discusión

La discusión final debe abordar:

- fortalezas reales del diseño modular;
- puntos de fricción en integración embebida;
- impacto de no tener todavía backend cloud en producción;
- criterio técnico para decidir la evolución de mecanismos de comunicación y backend.

6.8.1 Limitaciones del trabajo

Las limitaciones deben expresarse en términos de alcance de implementación, cobertura de pruebas y grado de madurez de integración.

6.8.2 Lecciones aprendidas

Como cierre de validación, este apartado debe recoger decisiones que hayan reducido riesgo técnico y pautas que mejoren la siguiente iteración del sistema.

Conclusiones

EL trabajo se consolida en torno a una idea principal: construir una arquitectura modular de captura inercial que pueda operar con nodo móvil y nodo embebido bajo contratos comunes.

7.1 Conclusiones principales

- El enfoque Android como plataforma operativa principal es viable para captura, gestión de sesiones y operación de campo.
- La separación por capas (adquisición, comunicación, backend, visualización/control) reduce acoplamiento y mejora mantenibilidad.
- La integración con ESP32 aporta una ruta realista para adquisición dedicada y validación de interoperabilidad.
- El uso de contratos comunes entre nodos evita bifurcar el sistema y facilita evolución tecnológica.
- El estado actual permite un flujo funcional local extremo a extremo y deja preparada la expansión a backend remoto.

7.2 Conclusión metodológica

La principal conclusión metodológica del TFG es que, en un proyecto de este alcance, resulta más sólido cerrar primero una arquitectura operativa por capas que comprometer desde el inicio una infraestructura definitiva.

Este criterio ha permitido:

- reducir riesgo técnico por integración progresiva;

- mantener trazabilidad entre decisiones y resultados;
- diferenciar claramente funcionalidades implementadas frente a líneas de evolución.

7.3 Implicaciones prácticas

Desde el punto de vista de ingeniería, la arquitectura propuesta permite:

- reutilizar la misma lógica de operación con distintos nodos de adquisición;
- introducir backend remoto sin rediseñar captura ni interfaz;
- evaluar tecnologías de comunicación por evidencia, no por compromiso previo.

7.4 Aportaciones

Las aportaciones del trabajo en su estado actual son:

- definición de una arquitectura ciberfísica modular para captura inercial;
- implementación de modo móvil operativo con gestión de sesiones;
- integración funcional Android-ESP32 para control y descarga de capturas;
- formalización de contratos de sesión, control y explotación;
- marco de validación orientado a interoperabilidad y operación.

7.5 Trabajo futuro

Las líneas de continuidad más relevantes son:

- integrar una primera capa backend para sincronización de sesiones y estado;
- ejecutar campaña de validación comparativa completa con protocolo homogéneo;
- mejorar trazabilidad automática (versionado de configuración y registro de incidencias);
- evaluar alternativas de comunicación externa si aportan mejoras medibles en operación;
- reforzar automatización de pruebas de integración entre app y nodo embebido.

7.5.1 Línea prioritaria inmediata

La prioridad inmediata es consolidar una primera versión funcional de capa backend y cerrar la validación de integración extremo a extremo sobre esa base.

Apéndices

Apéndice A

Material adicional

EN este apéndice se recoge material de apoyo para reproducir el entorno experimental y el flujo de trabajo técnico del proyecto.

A.1 Scripts principales

- `capture_http_imu.py`: recepción HTTP y guardado de sesiones.
- `process_imu_session.py`: procesado offline con métricas y visualización.
- `process_imu_step_by_step.py`: visualización por etapas del pipeline.

A.2 Comandos de referencia

A.2.1 Captura

```
1 python3 capture_http_imu.py --host 0.0.0.0 --port 8001 --seconds 30
```

A.2.2 Procesado rápido sin gráfica

```
1 python3 process_imu_session.py session_YYYYMMDD_HHMMSS_30s.csv  
   --no-plot
```

A.2.3 Procesado paso a paso con validación temporal

```
1 python3 process_imu_step_by_step.py session_YYYYMMDD_HHMMSS_30s.csv  
   \  
2 --mode step4 --g-fc 0.03 --fmin 0.02 --fmax 1.20 \  
3 --height-min-change 0.00005 --height-step-min 0.000001  
   --confirm-samples 6
```

A.3 Datos de prueba

Se trabajó con múltiples sesiones de 30 segundos en formato CSV, incluyendo capturas de reposo y capturas con movimiento vertical manual. La bitácora de resultados y ajustes se mantiene en `INFORME_IMU.md`.

Bibliografía
